

Data Compression Algorithms

Huffman Coding
Run Length Coding

Data Compression



In data compression is reducing the space required to storing a piece of data.

- We can replace recurrent elements in the data with a unique representation for each distinct element, which will require lesser space.
- Based on the unique representations we used to denote each element, the compressed data can be decompressed to get the original message

Encoding



Given a code (corresponding to some alphabet C) and a message the message is encoded by replacing the characters with the codewords

How can we represent a-f, using fixed length codes?

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

Encoding

Using fixed length codes:

6 characters $\rightarrow$ 3 bits are required for each code

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed length code						

Encoding



File size to store 100,000 characters? 300,000

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed length code	000	001	010	011	100	101

Encoding



Variable length codewords

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed length code	000	001	010	011	100	101
Variable length codes	0	101	100	111	1101	1100

Encoding

File size

$$= (45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4) * 1000$$
$$= 224\ 000$$

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed length code	000	001	010	011	100	101
Variable length codes	0	101	100	111	1101	1100

Decoding



Given an encoded message, decoding is the process of turning it back into the original message.

- A message is uniquely decodable if it can only be decoded in one way.
- The unique decipherability property is needed in order for a code to be useful.

Character	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed length code	000	001	010	011	100	101
Variable length codes	0	101	100	111	1101	1100

Decode the
sequence:
001011101

Prefix Codes



- Prefix codes are desirable because they simplify decoding.
 - In encoding; we just concatenate the codewords representing each character of the file
 - Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous.

Huffman Codes

Huffman codes are an optimal code of variable length codewords which can be used for lossless data compression.

Here, we consider the data to be a sequence of characters; Huffman's greedy algorithm uses a table giving how often each character occurs (**frequency**) to **build up an optimal** way of representing each character as a unique binary string (binary character code) which is called a **codeword**.

These can be used to compress data very effectively

- savings of 20% to 90% are typical, depending on the characteristics of the data being compressed.

Prefix Codes



Given a tree corresponding to the prefix codes, we can easily decode a character sequence

Let C be the alphabet, a set of n characters and that each character $c \in C$ is an object with attribute $c.freq$

The tree (T) for an optimal prefix code has

- exactly n leaves and $n - 1$ internal nodes

For each character c

- $d_T(c)$ denote the depth of the c 's node $\rightarrow$ length of the codeword for c

Prefix Codes



For each character c in the alphabet C ,

- the attribute $c.freq$ denote the frequency of c in the file
- $d_T(c)$ denote the depth of the c 's node $\rightarrow$ length of the codeword for c

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

The number of bits required to encode a file is,

This is also called the cost of the tree T .

✓ Huffman Coding

- The Huffman code, developed by D. Huffman in 1952, is a *minimum length code*
- This means that given the statistical distribution of the gray levels (the histogram), the Huffman algorithm will generate a code that is as close as possible to the *minimum bound, the entropy*

- The method results in an *unequal* (or *variable*) *length code*, where the size of the code words can vary
- For complex images, Huffman coding alone will typically reduce the file by 10% to 50% (1.1:1 to 1.5:1), but this ratio can be improved to 2:1 or 3:1 by preprocessing for irrelevant information removal

The Huffman algorithm can be described in five steps:

1. Find the gray level probabilities for the image by finding the histogram
2. Order the input probabilities (histogram magnitudes) from smallest to largest
3. Combine the smallest two by addition
4. GOTO step 2, until only two probabilities are left
5. By working backward along the tree, generate code by alternating assignment of 0 and 1

We have an image with 2-bits per pixel, giving four possible gray levels. The image is 10 rows by 10 columns.

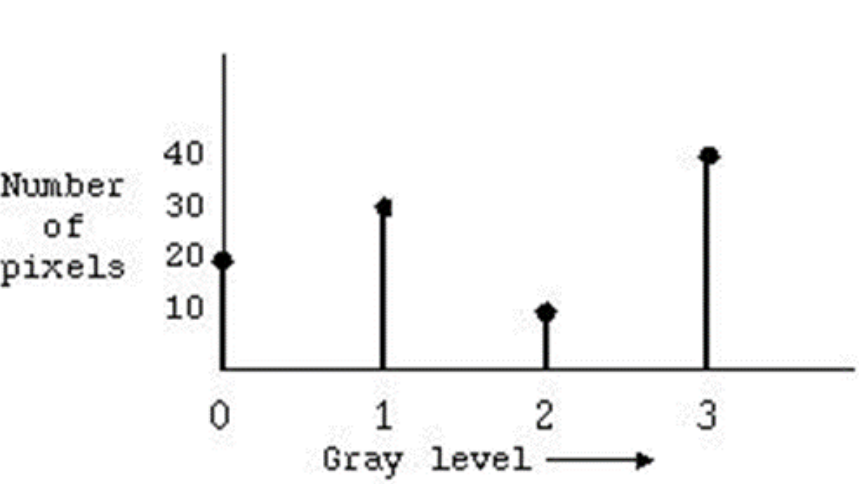
In Step 1 we find the histogram for the image. This is shown in Figure 10.2-2a, where we see that gray level 0 has 20 pixels, gray level 1 has 30 pixels, gray level 2 has 10 pixels, and gray level 3 has 40 pixels with the value. These are converted into probabilities by normalizing to the total number of pixels in the image.

Next, in step 2, the probabilities are ordered as in Figure 10.2-2b.

For step 3, we combine the smallest two by addition.

Step 4 repeats steps 2 and 3, where we reorder (if necessary) and add the two smallest probabilities as in Figure 10.2-2d. This step is repeated until only two values remain.

Figure 10.2-2: Huffman Coding Example

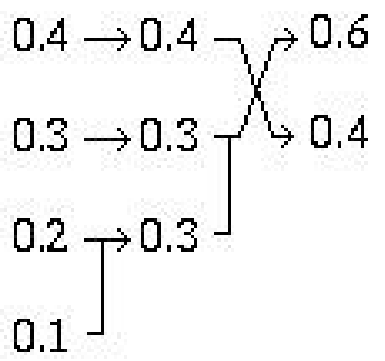
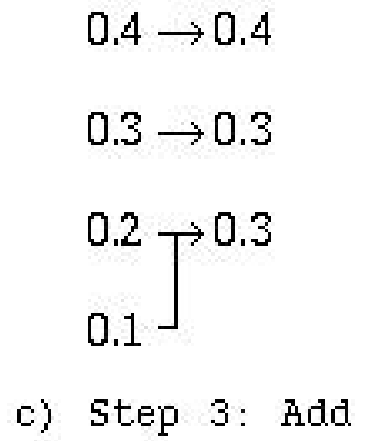
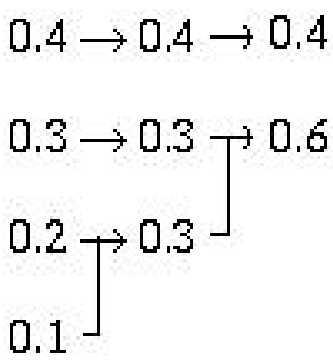


a) Step 1: Histogram

$$g_0 = \frac{20}{100} = 0.2$$
$$g_1 = \frac{30}{100} = 0.3$$
$$g_2 = \frac{10}{100} = 0.1$$
$$g_3 = \frac{40}{100} = 0.4$$

$g_3 \rightarrow 0.4$
 $g_1 \rightarrow 0.3$
 $g_0 \rightarrow 0.2$
 $g_2 \rightarrow 0.1$

b) Step 2: Order



d) Step 4: Reorder and add until only two values remain

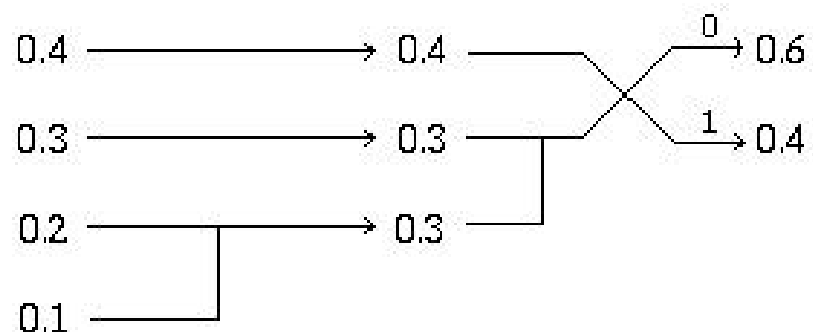
Since we have only two left in our example, we can continue to step 5 where the actual code assignment is made. The code assignment is shown in Figure 10.2-3. We start on the right-hand side of this tree and assign 0's and 1's, working our way back to the original probabilities.

Figure 10.2-3a shows the first assignment of 0 and 1. A 0 is assigned to the 0.6 branch, and a 1 to the 0.4 branch. In Figure 10.2-3b, the assigned 0 and 1 are brought back along the tree, and wherever a branch occurs the code is put on both branches.

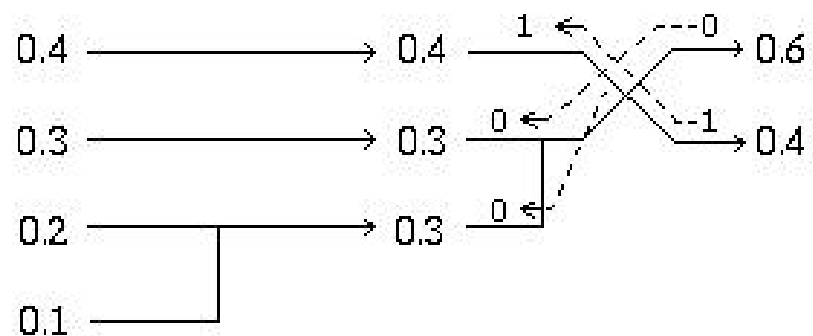
Now (Figure 10.2-3c), we assign the 0 and 1 to the branches labeled 0.3, appending to the existing code. Finally (Figure 10.2-3d), the codes are brought back one more level, and where the branch splits another assignment of 0 and 1 occurs (at the 0.1 and 0.2 branch).

Now we have the Huffman code for this image in Table 10-1.

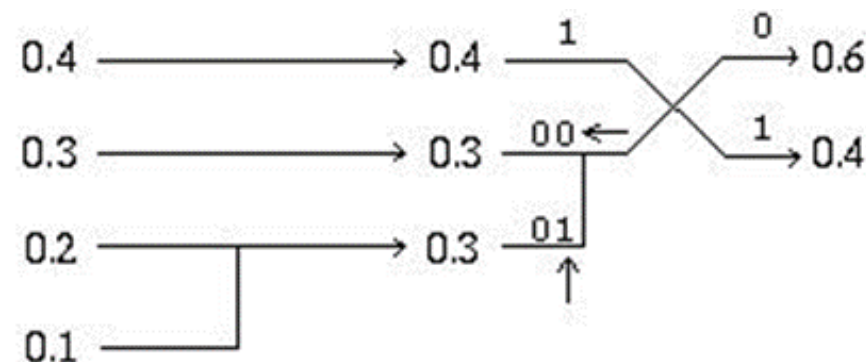
Figure 10.2-3: Huffman Coding Example, Step 5



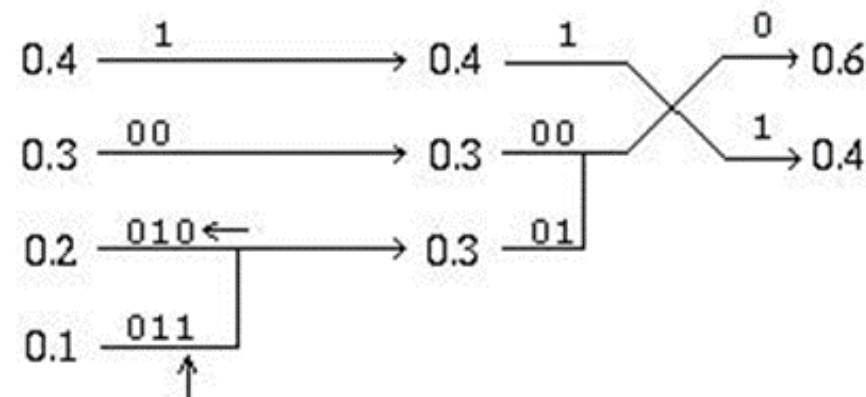
a) Assign 0 and 1 to the right-most probabilities



b) Bring 0 and 1 back along the tree



c) Append 0 and 1 to previously-added branches



d) Repeat the process until the original branch is labeled

TABLE 10-1

Original Gray Level (Natural Code)	Probability	Huffman code
$g_0: 00_2$	0.2	010_2
$g_1: 01_2$	0.3	00_2
$g_2: 10_2$	0.1	011_2
$g_3: 11_2$	0.4	1_2

EXAMPLE 10.2.4:

$$\begin{aligned} \text{Entropy} &= - \sum_{i=0}^3 p_i \log_2(p_i) \\ &= -[(0.2)\log_2(0.2) + (0.3)\log_2(0.3) + (0.1)\log_2(0.1) + (0.4)\log_2(0.4)] \\ &\approx 1.846 \text{ bits/pixel} \end{aligned}$$

(Note : $\log_2(x)$ can be found by taking $\log_{10}(x)$ and multiplying by 3.322)

$$\begin{aligned} L_{\text{ave}} &= \sum_{i=0}^{L-1} l_i p_i \\ &= 3(0.2) + 2(0.3) + 3(0.1) + 1(0.4) \\ &= 1.9 \text{ bits/pixel (Average length with Huffman code)} \end{aligned}$$

- In the example, we observe a 2.0 : 1.9 compression, which is about a 1.05 compression ratio, providing about 5% compression
- From the example we can see that the Huffman code is highly dependent on the histogram, so any preprocessing to simplify the histogram will help improve the compression ratio

✓ Run-Length Coding

- Run-length coding (RLC) works by counting adjacent pixels with the same gray level value called the *run-length*, which is then encoded and stored
- RLC works best for binary, two-valued, images
- RLC can also work with complex images that have been preprocessed by thresholding to reduce the number of gray levels to two

Run-Length Coding - Implementation

- RLC can be implemented in various ways, but the first step is to define the required parameters
- Horizontal RLC (counting along the rows) or vertical RLC (counting along the columns) can be used
- In basic horizontal RLC, the number of bits used for the encoding depends on the number of pixels in a row
- If the row has 2^n pixels, then the required number of bits is n , so that a run that is the length of the entire row can be encoded

EXAMPLE 10.2.5:

A 256x256 image requires 8-bits, since $2^8 = 256$.

EXAMPLE 10.2.6:

A 512x512 image requires 9-bits, since $2^9 = 512$.

- The next step is to define a convention for the first RLC number in a row – does it represent a run of 0's or 1's?

EXAMPLE 10.2.7:

The image is an 8x8 binary image, which requires 3 bits for each run-length coded word. In the actual image file are stored 1's and 0's, although upon display the 1's become 255 (white) and the 0's are 0 (black). To apply RLC to this image, using horizontal RLC:

0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0
0	1	1	0	0	0	0	0
0	1	1	1	1	1	0	0
0	1	1	1	0	0	1	0
0	0	1	0	0	1	1	0
1	1	1	1	0	1	0	0
0	0	0	0	0	0	0	0

EXAMPLE 10.2.7 (contd):

The RLC numbers are:

First row: 8

Second row: 0, 4, 4

Third row: 1, 2, 5

Fourth row: 1, 5, 2

Fifth row: 1, 3, 2, 1, 1

Sixth row: 2, 1, 2, 2, 1

Seventh row: 0, 4, 1, 1, 2

Eighth row: 8

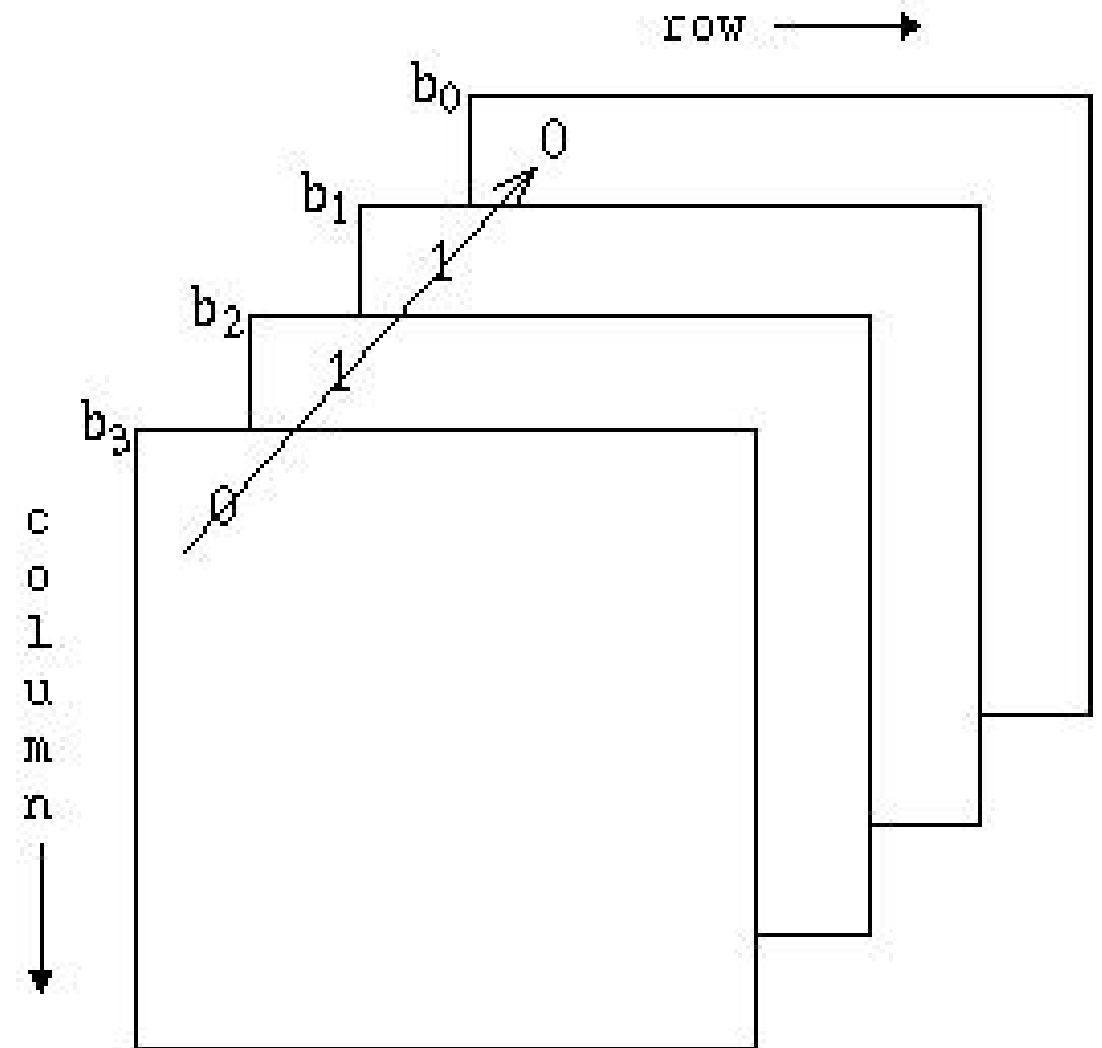
Note that in the second and seventh rows, the first RLC number is 0, since we are using the convention that the first number corresponds to the number of zeros in a run

- *Bitplane-RLC* : A technique which involves extension of basic RLC method to gray level images, by applying basic RLC to each bit-plane independently
- For each binary digit in the gray level value, an image plane is created, and this image plane (a string of 0's and 1's) is then encoded using RLC

Figure 10.2-4: Bit Plane Run-length Coding

b_3	b_2	b_1	b_0
0	0	0	0
0	0	0	1
0	0	1	0
.	.	.	.
.	.	.	.
.	.	.	.
0	1	1	0
1	1	1	1

a) 4 bits/pixel designation



b) bit-planes $\longrightarrow$ b_3 b_2 b_1 b_0
0 1 1 0

- Typical compression ratios of 0.5 to 1.2 are achieved with complex 8-bit monochrome images
- Thus without further processing, this is not a good compression technique for complex images
- Bitplane-RLC is most useful for simple images, such as graphics files, where much higher compression ratios are achieved

- The compression results using this method can be improved by preprocessing to reduce the number of gray levels, but then the compression is *not lossless*
- With lossless bitplane RLC we can improve the compression results by taking our original pixel data (in *natural code*) and mapping it to a *Gray code* (named after Frank Gray), where adjacent numbers differ in only one bit
- As the adjacent pixel values are highly correlated, adjacent pixel values tend to be relatively close in gray level value, and this can be problematic for RLC

Figure 10.2-5: Gray Code

↓

Decimal	4-bit Natural Code	4-bit Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

a) Gray code versus natural code

```

7:  0  1  1  1
   ↓  ↓  ↓  ↓  ↓
8:  1  0  0  0

```

b) The natural code transition of 7 to 8 changes all 4 bits

EXAMPLE 10.2.8:

In Figure 10.2-5 is shown the 4-bit Gray code and the natural binary code. The Gray code, by definition, only has one bit changing in adjacent codes. However, in, for example, the 7 to 8 transition with the natural code, all 4 bits change:

<i>Natural Code</i>	<i>Gray Code</i>
0111	0100
↓↓↓↓	↓↓↓↓
1000	1100

- When a situation such as the above example occurs, each bitplane experiences a transition, which adds a code for the run in each bitplane
- However, with the Gray code, only one bitplane experiences the transition, so it only adds one extra code word
- By preprocessing with a Gray code we can achieve about a 10% to 15% increase in compression with bitplane-RLC for typical images

Thank you!